```
"""
COBOL Dependency Analyzer
Scans COBOL source files (.CIC, .CPY, .MPS, .SRC) and builds a dependency
graph based on COPY, CALL, and other inter-file references.
"""

import re
import os
from pathlib import Path
from collections import defaultdict
from dataclasses import dataclass, field


# ---------------------------------------------------------------------------
# Patterns
# ---------------------------------------------------------------------------

# Each entry: (dependency_type, compiled_regex)
# Regex rules:
#   - COBOL is case-insensitive; we compile with re.IGNORECASE
#   - Fixed-format COBOL: columns 1-6 are sequence/indicator area; column 7
#     is the indicator (*  = comment, /  = page eject, D = debug line).
#     We skip those lines before matching.
#   - Free-format COBOL (columns 8+) is also handled since we strip leading
#     whitespace after skipping the indicator column.

DEPENDENCY_PATTERNS: list[tuple[str, re.Pattern]] = [
    # COPY <member> [OF/IN <library>] [REPLACING ...]
    # Handles: COPY MYFILE, COPY MYFILE OF MYLIB, COPY 'MYFILE'
    (
        "COPY",
        re.compile(
            r"^\s{0,6}"          # optional sequence area (cols 1-6 may be blank)
            r"(?!\*)"            # not a comment line (col 7 = *)
            r"\s*"               # indent in Area A/B
            r"COPY\s+"
            r"(?P<member>['\"]?[\w\$\-\.#@]+['\"]?)"  # member name (optional quo
            r"(?:\s+(?:OF|IN)\s+(?P<library>[\w\$\-\.#@]+))?",  # optional OF/IN
            re.IGNORECASE | re.MULTILINE,
        ),
    ),
    # CALL '<literal>' / CALL identifier
    # Static calls use a string literal; dynamic calls use an identifier.
    # We capture both but tag them differently.
```

```
    (
        "CALL",
        re.compile(
            r"^\s{0,6}(?!\*)\s*"
            r"CALL\s+"
            r"(?P<target>['\"][\w\$\-\.#@]+['\"]|[\w\$\-\.#@]+)",
            re.IGNORECASE | re.MULTILINE,
        ),
    ),
    # EXEC CICS LINK PROGRAM(<name>)   — common in CIC files
    (
        "EXEC CICS LINK",
        re.compile(
            r"EXEC\s+CICS\s+LINK\s+PROGRAM\s*\(\s*(?P<program>['\"]?[\w\$\-\.#@]+
            re.IGNORECASE | re.MULTILINE,
        ),
    ),
    # EXEC CICS XCTL PROGRAM(<name>)
    (
        "EXEC CICS XCTL",
        re.compile(
            r"EXEC\s+CICS\s+XCTL\s+PROGRAM\s*\(\s*(?P<program>['\"]?[\w\$\-\.#@]+
            re.IGNORECASE | re.MULTILINE,
        ),
    ),
    # EXEC CICS START TRANSID(<name>)  — less direct but still a dependency
    (
        "EXEC CICS START",
        re.compile(
            r"EXEC\s+CICS\s+START\s+.*?TRANSID\s*\(\s*(?P<transid>['\"]?[\w\$\-\.
            re.IGNORECASE | re.MULTILINE | re.DOTALL,
        ),
    ),
]

TARGET_EXTENSIONS = {".CIC", ".CPY", ".MPS", ".SRC"}


# ---------------------------------------------------------------------------
# Data structures
# ---------------------------------------------------------------------------

@dataclass
class Dependency:
    dep_type: str          # e.g. "COPY", "CALL", "EXEC CICS LINK"
    raw_target: str        # exactly as found in source (may include quotes)
    resolved_target: str   # uppercased, stripped of quotes
```

```
    library: str | None    # only set for COPY … OF/IN <library>
    line_number: int


@dataclass
class FileAnalysis:
    path: Path
    extension: str
    dependencies: list[Dependency] = field(default_factory=list)
    errors: list[str] = field(default_factory=list)


# ---------------------------------------------------------------------------
# Core parsing
# ---------------------------------------------------------------------------

def _strip_quotes(value: str) -> str:
    return value.strip("'\"")


def _is_comment_line(line: str) -> bool:
    """
    In fixed-format COBOL column 7 (index 6) is '*' or '/' for comments.
    Free-format files won't have sequence numbers, so this heuristic is
    conservative: only skip if the line is long enough and col 7 matches.
    """
    if len(line) >= 7 and line[6] in ("*", "/", "d", "D"):
        return True
    # Also skip lines whose *stripped* content starts with * (some editors
    # omit the sequence area entirely).
    stripped = line.lstrip()
    return stripped.startswith("*") or stripped.startswith("/")


def parse_file(path: Path) -> FileAnalysis:
    analysis = FileAnalysis(path=path, extension=path.suffix.upper())
    try:
        # Try UTF-8 first, fall back to latin-1 (common for mainframe exports)
        try:
            source = path.read_text(encoding="utf-8")
        except UnicodeDecodeError:
            source = path.read_text(encoding="latin-1")

        lines = source.splitlines()

        for dep_type, pattern in DEPENDENCY_PATTERNS:
            for match in pattern.finditer(source):
```

```
                # Determine line number from match start position
                line_no = source[: match.start()].count("\n") + 1

                # Skip if the matched line is a comment
                matched_line = lines[line_no - 1] if line_no <= len(lines) else ''
                if _is_comment_line(matched_line):
                    continue

                # Extract named groups depending on pattern
                groups = match.groupdict()
                raw_target = (
                    groups.get("member")
                    or groups.get("target")
                    or groups.get("program")
                    or groups.get("transid")
                    or ""
                )
                if not raw_target:
                    continue

                resolved = _strip_quotes(raw_target).upper()
                library = groups.get("library")
                if library:
                    library = _strip_quotes(library).upper()

                analysis.dependencies.append(
                    Dependency(
                        dep_type=dep_type,
                        raw_target=raw_target,
                        resolved_target=resolved,
                        library=library,
                        line_number=line_no,
                    )
                )

    except OSError as exc:
        analysis.errors.append(str(exc))

    return analysis


# ---------------------------------------------------------------------------
# Directory scan
# ---------------------------------------------------------------------------

def scan_directory(
    root: str | Path,
```

```
    recursive: bool = True,
    extensions: set[str] | None = None,
) -> list[FileAnalysis]:
    """
    Scan *root* for COBOL source files and parse each one.

    Parameters
    ----------
    root        : Directory to scan.
    recursive   : If True, walk subdirectories.
    extensions  : Set of upper-case extensions to include (default: TARGET_EXTENS
    """
    root = Path(root)
    exts = extensions or TARGET_EXTENSIONS

    glob_pattern = "**/*" if recursive else "*"
    candidates = [p for p in root.glob(glob_pattern) if p.is_file()]
    target_files = [p for p in candidates if p.suffix.upper() in exts]

    return [parse_file(p) for p in sorted(target_files)]


# ---------------------------------------------------------------------------
# Dependency graph helpers
# ---------------------------------------------------------------------------

def build_dependency_graph(
    analyses: list[FileAnalysis],
) -> dict[str, list[dict]]:
    """
    Returns a dict mapping each source file name (stem, upper-case) to a list
    of dependency records.

    Example output:
        {
            "MAINPROG": [
                {"dep_type": "COPY", "target": "COPYBOOK1", "library": None,  "li
                {"dep_type": "CALL", "target": "SUBPROG",   "library": None,  "li
            ],
            ...
        }
    """
    graph: dict[str, list[dict]] = defaultdict(list)
    for analysis in analyses:
        key = analysis.path.stem.upper()
        for dep in analysis.dependencies:
            graph[key].append(
```

```
                {
                    "dep_type": dep.dep_type,
                    "target": dep.resolved_target,
                    "library": dep.library,
                    "line": dep.line_number,
                    "source_file": str(analysis.path),
                }
            )
    return dict(graph)


def find_dependents(
    graph: dict[str, list[dict]], target_name: str
) -> list[str]:
    """Return all files that depend on *target_name* (reverse lookup)."""
    target = target_name.upper()
    return [src for src, deps in graph.items() if any(d["target"] == target for d


def find_transitive_dependencies(
    graph: dict[str, list[dict]], start: str, _visited: set[str] | None = None
) -> set[str]:
    """Return all transitive dependencies of *start* (depth-first)."""
    if _visited is None:
        _visited = set()
    start = start.upper()
    if start in _visited:
        return _visited
    _visited.add(start)
    for dep in graph.get(start, []):
        find_transitive_dependencies(graph, dep["target"], _visited)
    _visited.discard(start)  # remove self; return only dependencies
    return _visited


# ---------------------------------------------------------------------------
# Reporting
# ---------------------------------------------------------------------------

def print_report(analyses: list[FileAnalysis], verbose: bool = False) -> None:
    graph = build_dependency_graph(analyses)

    total_files = len(analyses)
    total_deps = sum(len(a.dependencies) for a in analyses)
    error_files = [a for a in analyses if a.errors]

    print("=" * 70)
```

```
    print("COBOL DEPENDENCY ANALYSIS REPORT")
    print("=" * 70)
    print(f"Files scanned : {total_files}")
    print(f"Total deps    : {total_deps}")
    print(f"Files w/errors: {len(error_files)}")
    print()

    if error_files:
        print("--- ERRORS ---")
        for a in error_files:
            for err in a.errors:
                print(f"  {a.path}: {err}")
        print()

    print("--- DEPENDENCIES BY FILE ---")
    for analysis in analyses:
        rel = analysis.path
        if not analysis.dependencies:
            if verbose:
                print(f"  {rel}  (no dependencies found)")
            continue

        print(f"\n  {rel}")
        by_type: dict[str, list[Dependency]] = defaultdict(list)
        for dep in analysis.dependencies:
            by_type[dep.dep_type].append(dep)

        for dep_type, deps in sorted(by_type.items()):
            print(f"    [{dep_type}]")
            for dep in deps:
                lib_part = f"  (library: {dep.library})" if dep.library else ""
                print(f"      line {dep.line_number:>5}: {dep.resolved_target}{li

    print()
    print("--- REVERSE LOOKUP (who depends on X) ---")
    all_targets = {d["target"] for deps in graph.values() for d in deps}
    for target in sorted(all_targets):
        dependents = find_dependents(graph, target)
        print(f"  {target:30s} <-- {', '.join(sorted(dependents))}")

    print("=" * 70)


def export_csv(analyses: list[FileAnalysis], output_path: str | Path) -> None:
    """Write a flat CSV with one row per dependency."""
    import csv
```

```
    output_path = Path(output_path)
    with output_path.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.writer(fh)
        writer.writerow(
            ["source_file", "source_stem", "extension", "dep_type", "target", "li
        )
        for analysis in analysis:
            for dep in analysis.dependencies:
                writer.writerow(
                    [
                        str(analysis.path),
                        analysis.path.stem.upper(),
                        analysis.extension,
                        dep.dep_type,
                        dep.resolved_target,
                        dep.library or "",
                        dep.line_number,
                    ]
                )
    print(f"CSV written to: {output_path}")


def export_dot(analyses: list[FileAnalysis], output_path: str | Path) -> None:
    """
    Write a Graphviz DOT file for visualising the dependency graph.
    Open with: dot -Tpng dependencies.dot -o dependencies.png
    """
    graph = build_dependency_graph(analyses)
    output_path = Path(output_path)

    with output_path.open("w", encoding="utf-8") as fh:
        fh.write("digraph cobol_deps {\n")
        fh.write('    rankdir=LR;\n')
        fh.write('    node [shape=box fontname="Courier"];\n\n')

        for src, deps in sorted(graph.items()):
            for dep in deps:
                label = dep["dep_type"]
                fh.write(f'    "{src}" -> "{dep["target"]}" [label="{label}"];\n'

        fh.write("}\n")
    print(f"DOT file written to: {output_path}")


# ---------------------------------------------------------------------------
# CLI entry point
# ---------------------------------------------------------------------------
```

```
def main():
    import argparse

    parser = argparse.ArgumentParser(
        description="Analyse COBOL source files for inter-file dependencies."
    )
    parser.add_argument("directory", help="Root directory to scan")
    parser.add_argument(
        "--no-recursive", dest="recursive", action="store_false", default=True,
        help="Do not recurse into subdirectories",
    )
    parser.add_argument(
        "--extensions", nargs="+", default=None,
        help="File extensions to scan (default: CIC CPY MPS SRC). "
             "Provide without the dot, e.g. --extensions CIC CPY",
    )
    parser.add_argument("--csv", metavar="PATH", help="Export flat CSV to PATH")
    parser.add_argument("--dot", metavar="PATH", help="Export Graphviz DOT to PAT
    parser.add_argument(
        "--verbose", "-v", action="store_true",
        help="Show files with no dependencies in the report",
    )
    parser.add_argument(
        "--transitive", metavar="FILE_STEM",
        help="Print all transitive dependencies of the given file stem",
    )

    args = parser.parse_args()

    exts = (
        {e.upper() if e.startswith(".") else f".{e.upper()}" for e in args.extens
        if args.extensions
        else None
    )

    analyses = scan_directory(args.directory, recursive=args.recursive, extension

    print_report(analyses, verbose=args.verbose)

    if args.csv:
        export_csv(analyses, args.csv)

    if args.dot:
        export_dot(analyses, args.dot)

    if args.transitive:
```

```
        graph = build_dependency_graph(analyses)
        transitive = find_transitive_dependencies(graph, args.transitive)
        print(f"\nTransitive dependencies of {args.transitive.upper()}:")
        for dep in sorted(transitive):
            print(f"  {dep}")


if __name__ == "__main__":
    main()
```